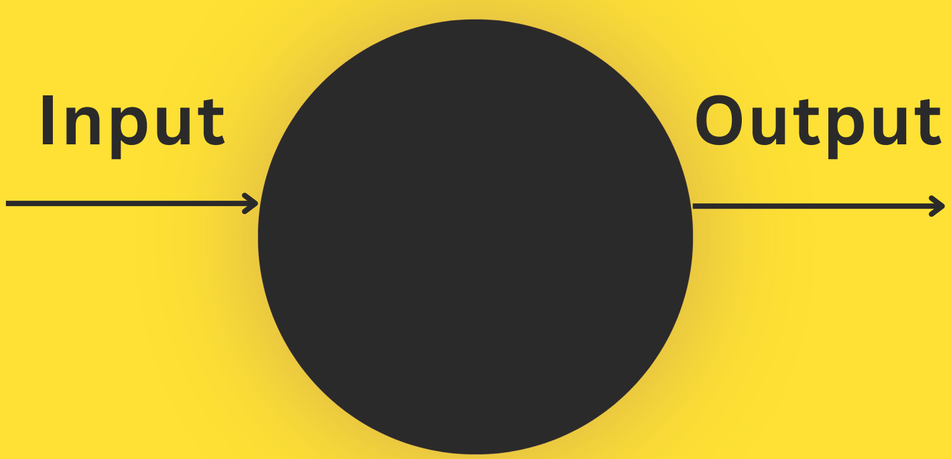
MULTI-AGENT ORCHESTRATION PATTERNS

Sandipan Bhaumik
Data & AI Tech Lead
Databricks

April 2026

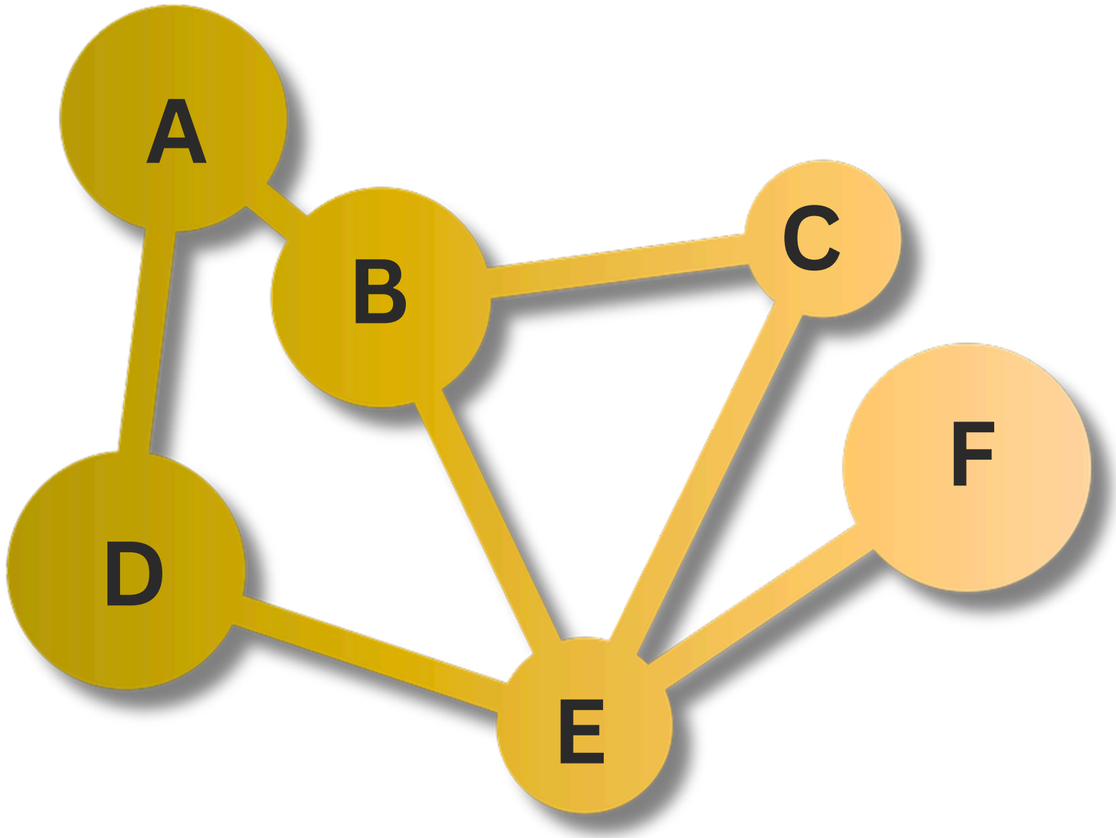
AGENDA

- **PROBLEM**
  War story, why complexity explodes
- **PATTERNS**
  Coordination
  State management (immutable)
  Failure recovery
- **IMPLEMENTATION**
  Production architecture



ONE AGENT WORKS

One agent is a feature



FIVE AGENTS ARE CHAOS

Five agents is a distributed systems problem

WAR STORY

THE RACE CONDITION

Week 1:
Credit Agent Deployed

Week 2:
5 Agents Live in Production

Week 3 - 5:
Firefighting

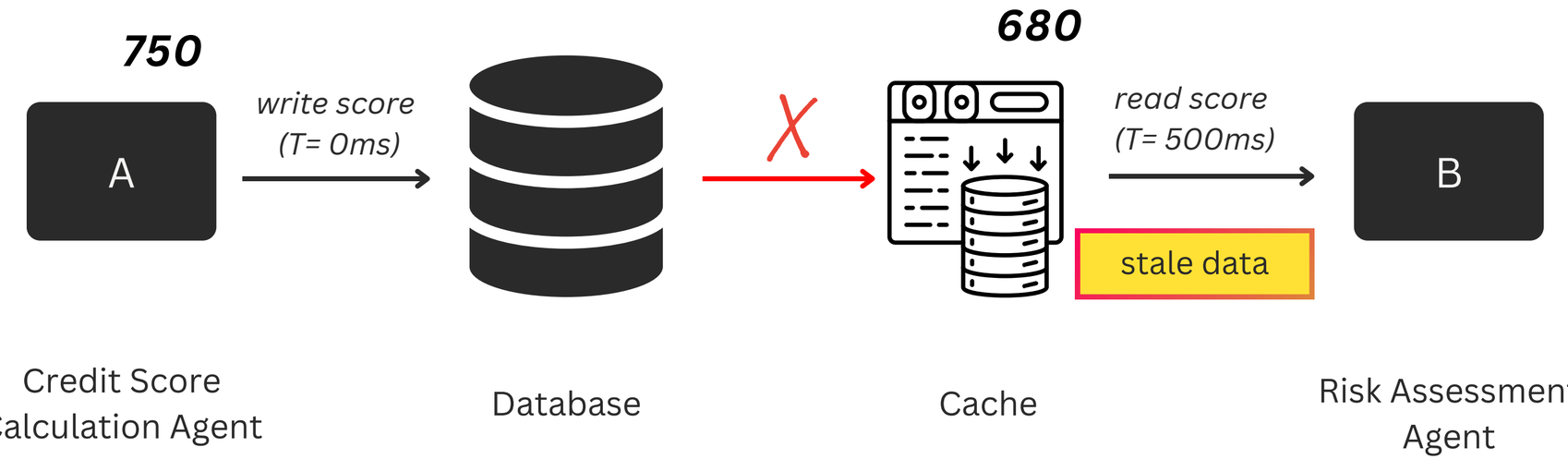
Incorrect Risk Ratings

Incorrectly flagged customers

Delayed Value

WAR STORY

THE BUG

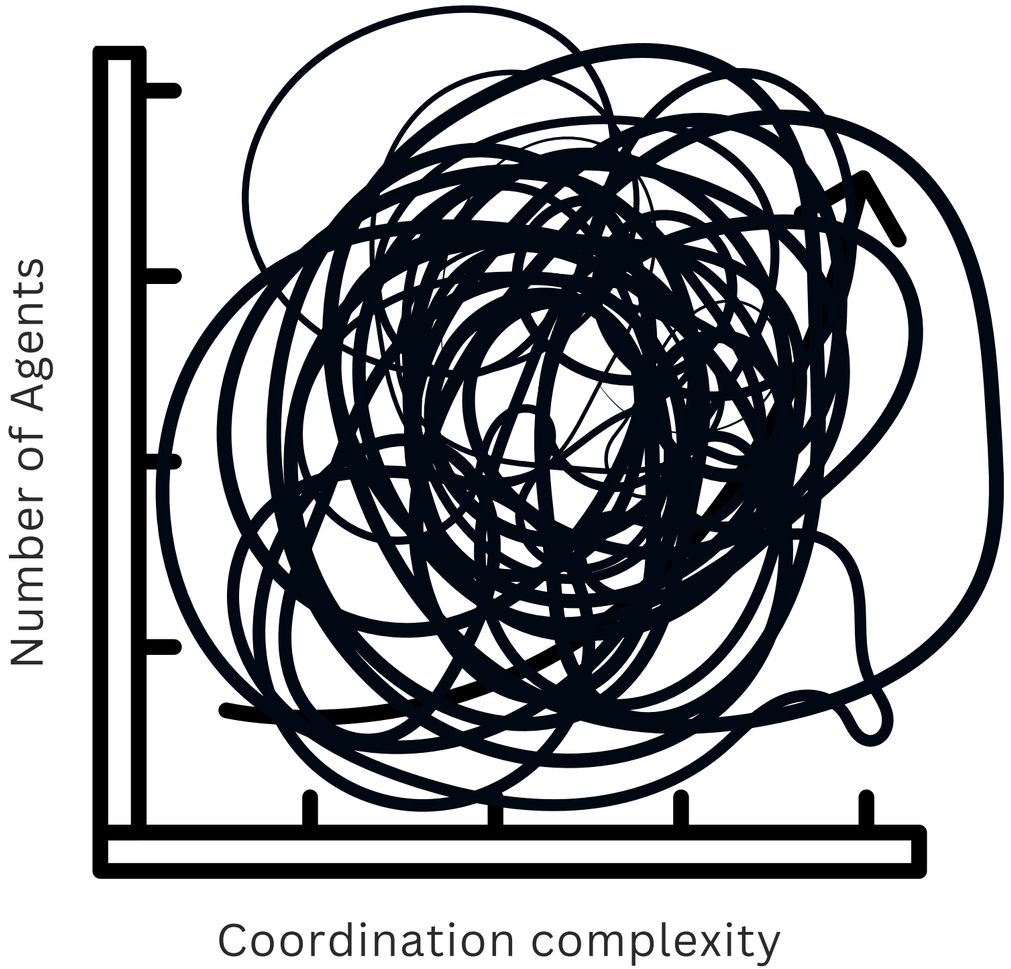


Agent B is reading stale version of the data - that leads to wrong decisions

THE PROBLEM WASN’T THE LLM

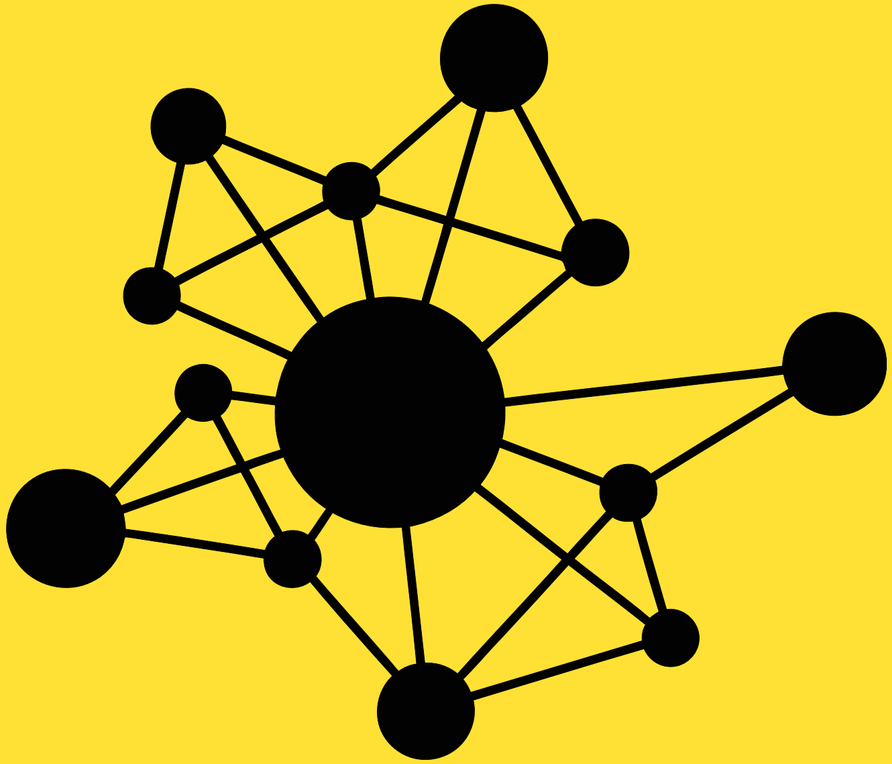
We built a distributed system…

…without
distributed system thinking



THE COMPLEXITY CURVE

5 agents = 25x complex



PATTERNS

THE FUNDAMENTAL QUESTION

How Should Your Agents Coordinate?

VS

CHOREOGRAPHY

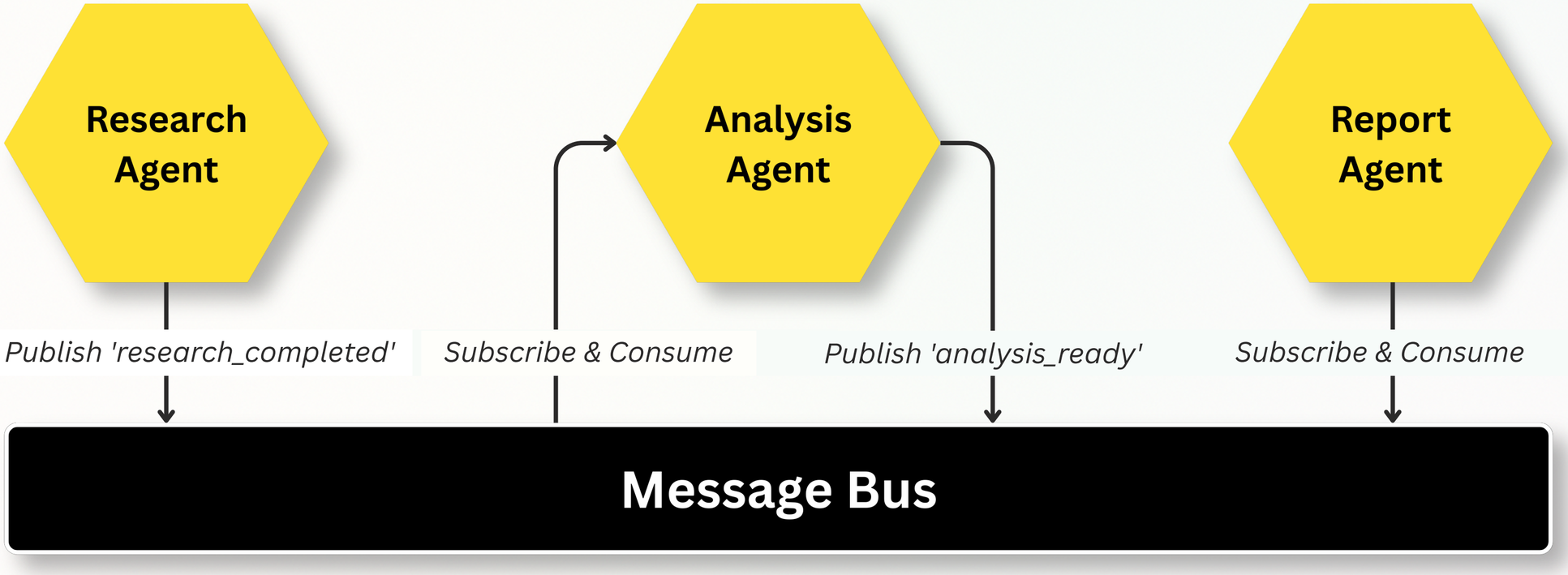
Decentralized. Event-driven. Autonomous

ORCHESTRATION

Centralized. Coordinated. Controlled

CHOREOGRAPHY

EVENT-DRIVEN COORDINATION



CHOREOGRAPHY

WHEN TO USE AND AVOID

Loosely Coupled Workflows

High Agent Autonomy needed

Frequently Added Agents

Strong Observability Infrastructure

Complex Dependencies

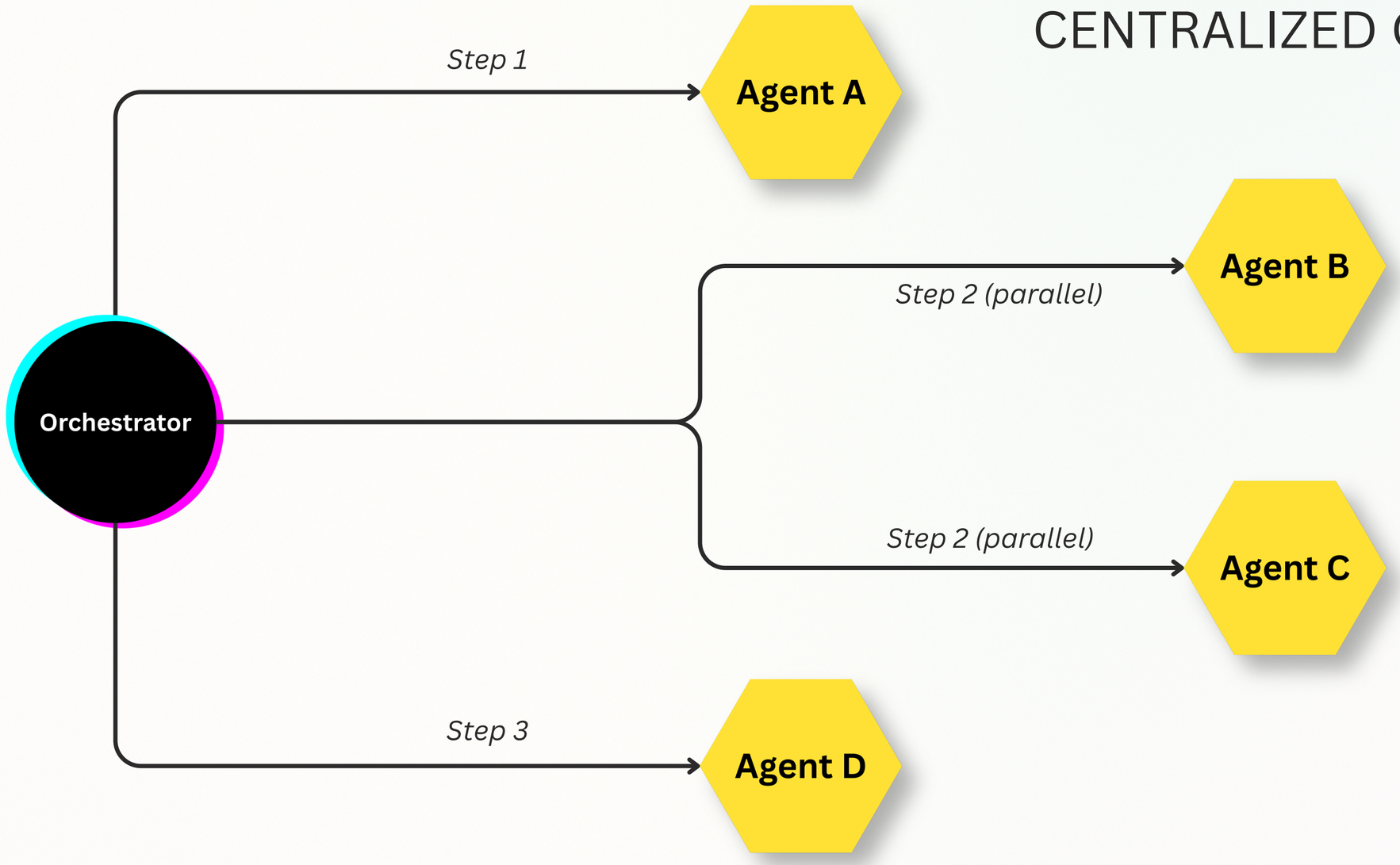
Need Centralized Rollback

Weak Observability

Debugging is Difficult

ORCHESTRATION

CENTRALIZED COORDINATION



ORCHESTRATION

WHEN TO USE AND AVOID

Complex Dependencies

Rollback Capability Required

Need Centralized Control

Stable Workflows

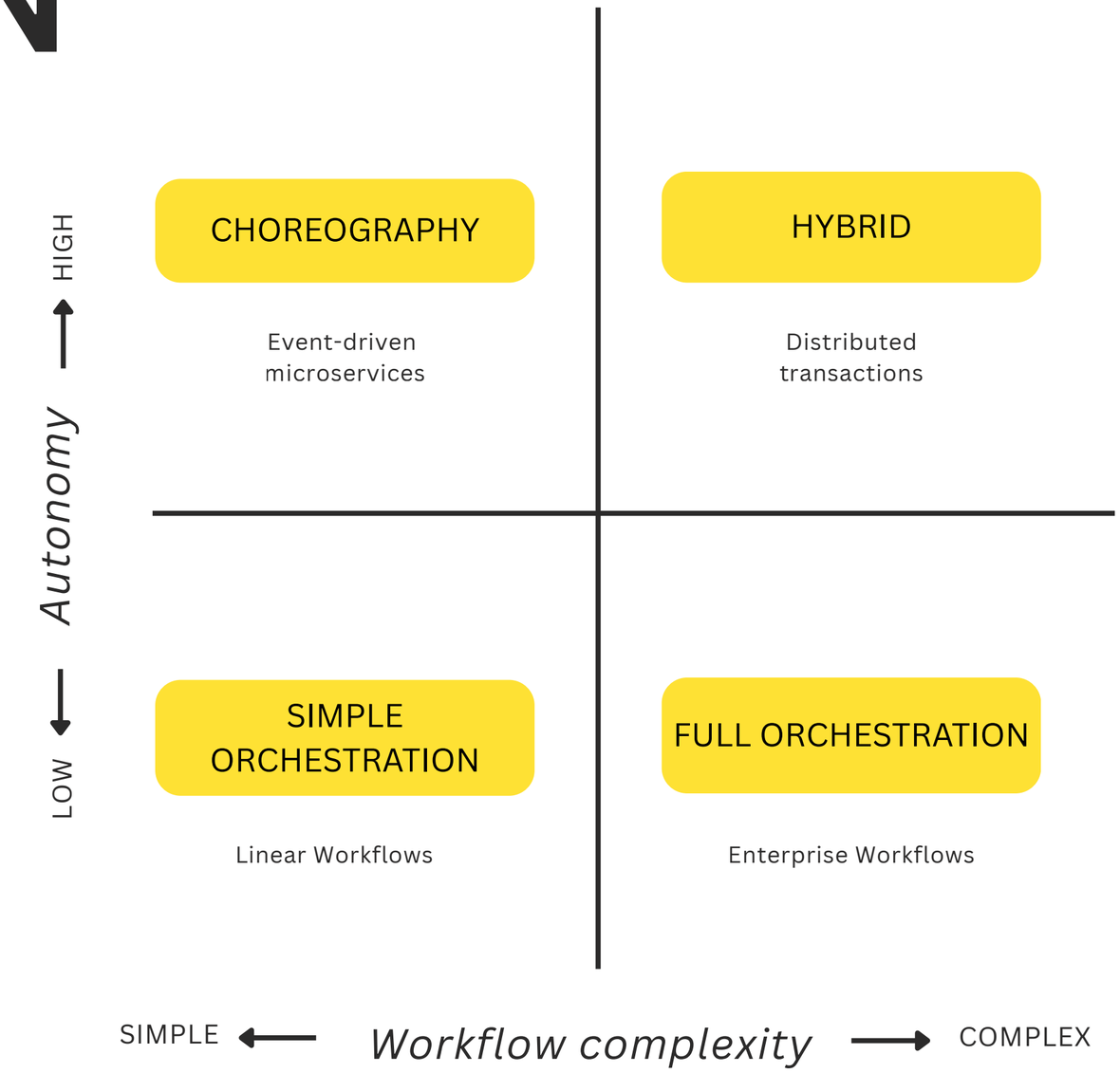
High Autonomy Required

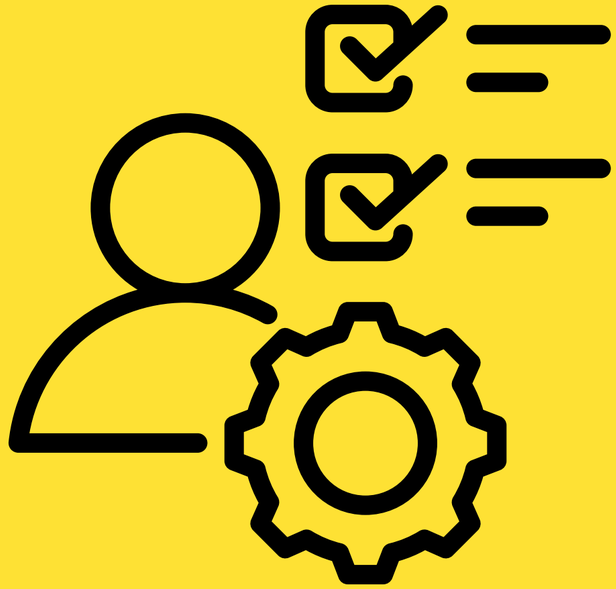
Can’t Tolerate Bottleneck

Constantly Changing Workflows

Need Distributed Scaling

DECISION MATRIX

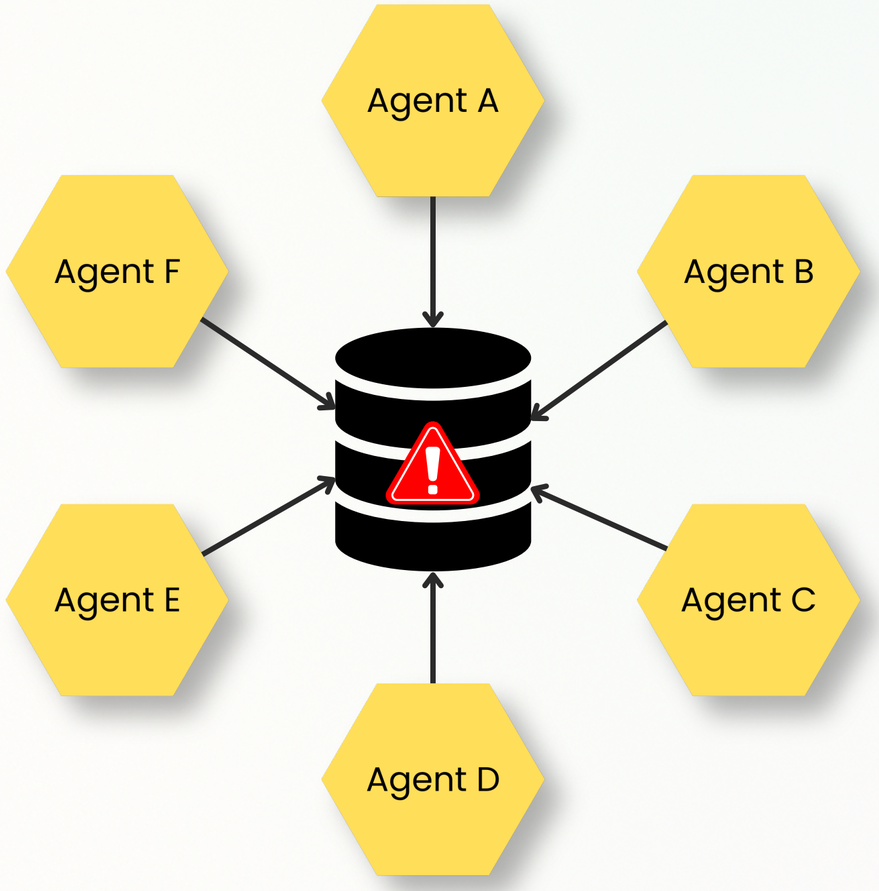




STATE MANAGEMENT

ANTI-PATTERN

SHARED MUTABLE STATE



CORRECT PATTERN

IMMUTABLE STATE SNAPSHOTS

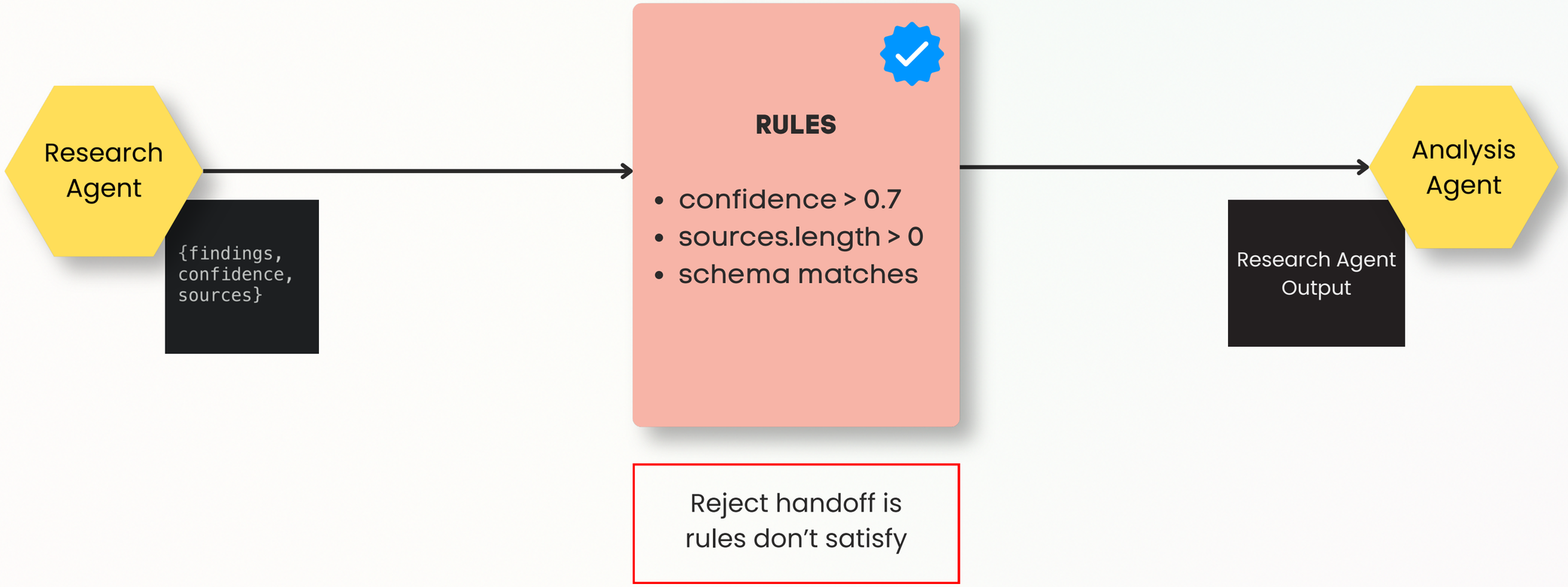
State snapshots can be logged to append-only storage for audit/replay, but never shared for read/write

STATE HANDOFF

```python
@dataclass(frozen=True)  # ← Immutable
class AgentState:
    version: int
    data: Dict
    created_by: str

def handoff(state: AgentState) -> AgentState:
    validated = validate_schema(state)  # ← Contract

    next_state = AgentState(
        version=state.version + 1,      # ← Versioned
        data=validated.data,
        created_by=next_agent.name
    )

    return next_agent.execute(next_state)
```

DATA CONTRACTS

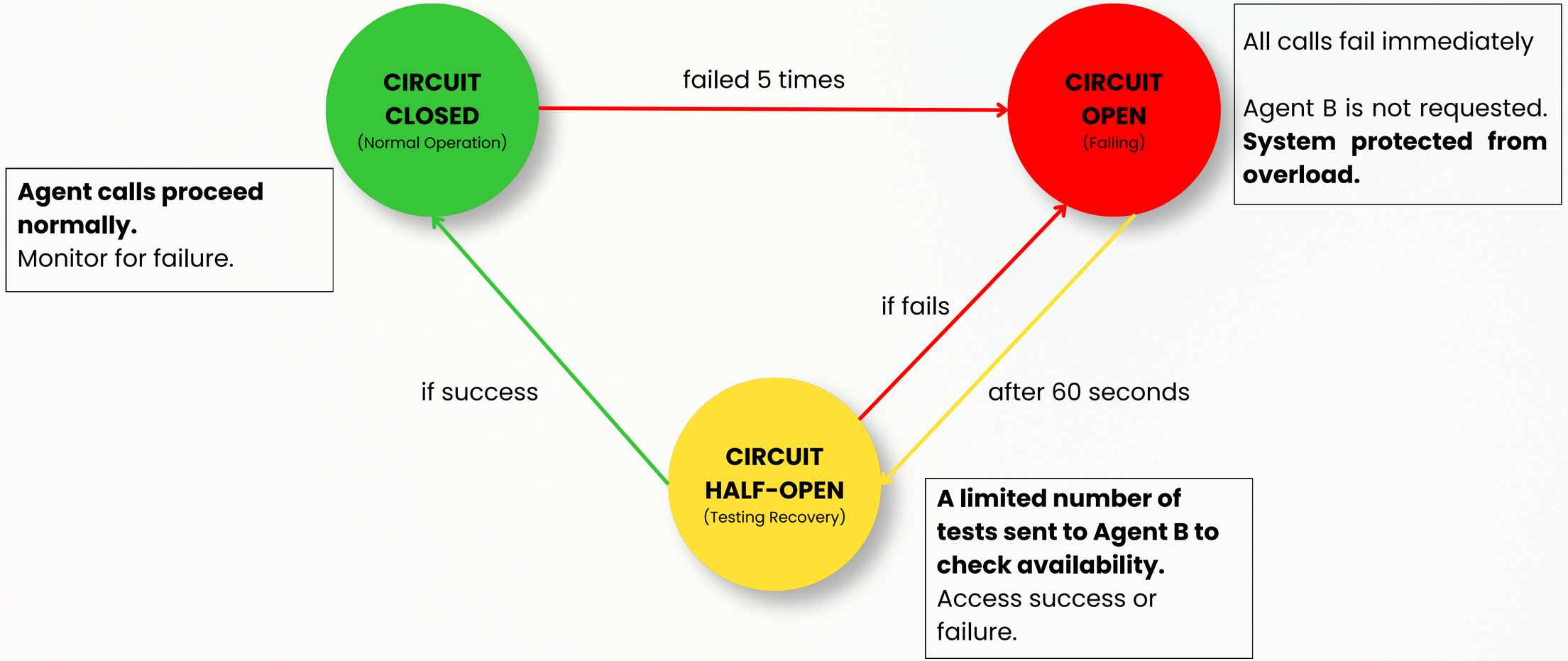
WHY THEY MATTER?



FAILURE & RECOVERY

CIRCUIT BREAKER PATTERN

FAIL FAST, RECOVER GRACEFULLY



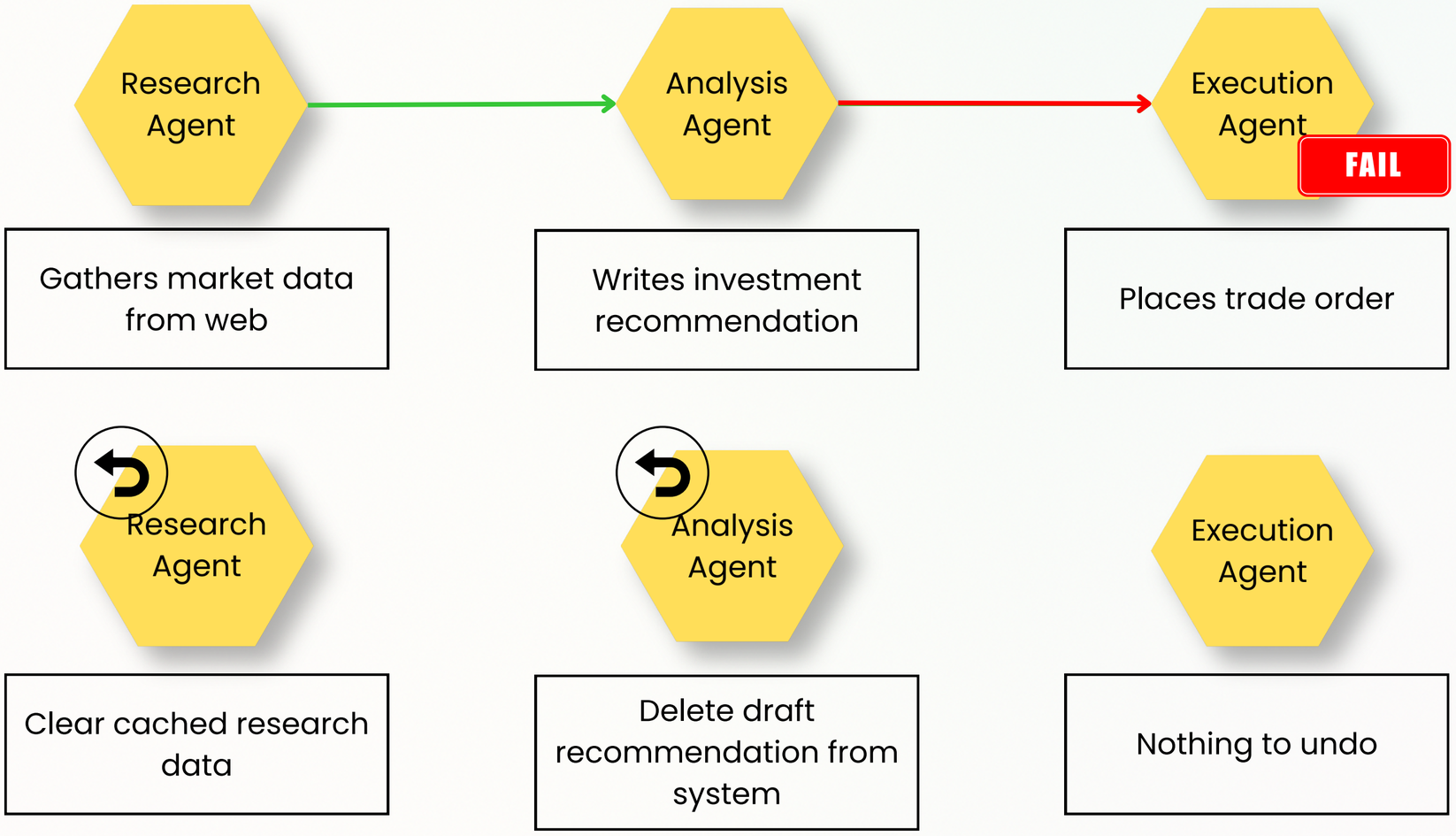
CIRCUIT BREAKER

IMPLEMENTATION

```python
class CircuitBreaker:
    def __init__(self, failure_threshold=5, timeout=60):
        self.failure_count = 0
        self.state = "CLOSED"  # CLOSED | OPEN | HALF_OPEN

    def call(self, agent_fn):
        if self.state == "OPEN":
            if self._should_test():
                self.state = "HALF_OPEN"
            else:
                raise CircuitOpenError()

        try:
            result = agent_fn()
            self._on_success()  # Reset to CLOSED
            return result
        except Exception:
            self._on_failure()  # Increment, maybe OPEN
            raise
```

COMPENSATION PATTERN

ROLLBACK WHEN FAILURE HAPPENS MID-WORKFLOW

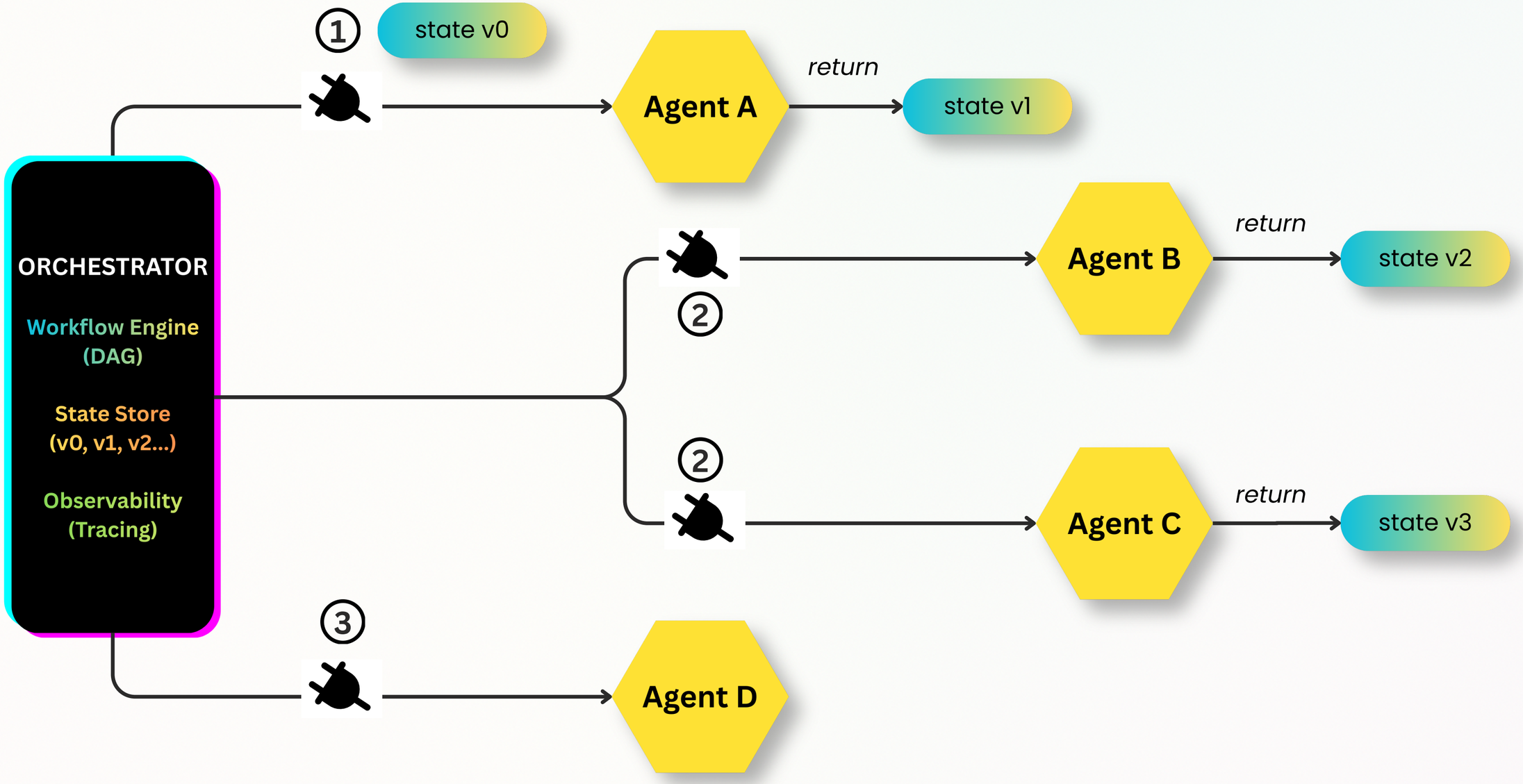


COMPESATION (SAGA) PATTERN

IMPLEMENTATION

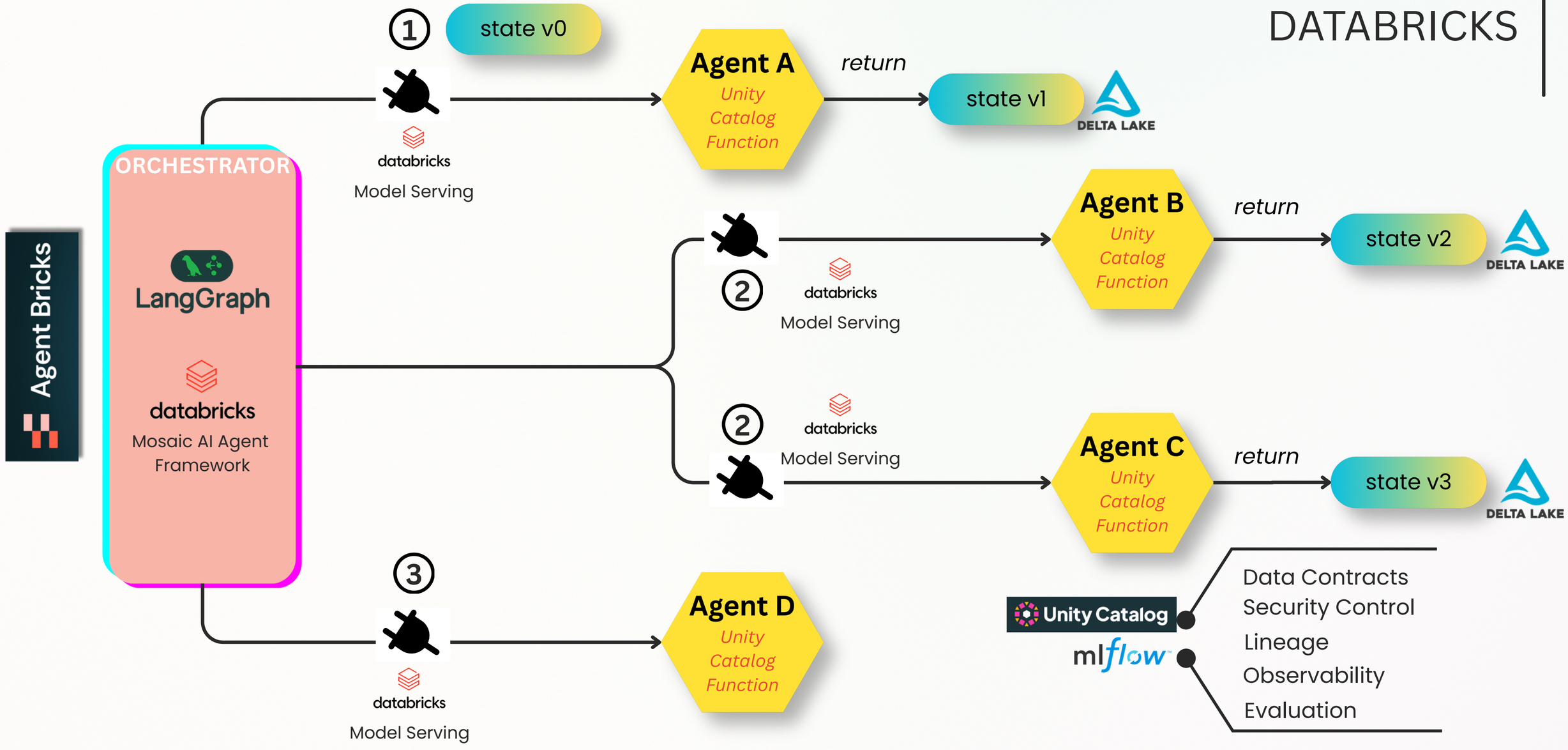
```python
class CompensatingAgent:
    def execute(self, state):
        # Normal operation
        return result

    def compensate(self, state):
        # Undo the operation
        return rollback_result

# Orchestrator tracks what executed
executed_agents = []
try:
    for agent in workflow:
        result = agent.execute(state)
        executed_agents.append((agent, result))
except AgentFailure:
    # Compensate in reverse order
    for agent, result in reversed(executed_agents):
        agent.compensate(result)
```

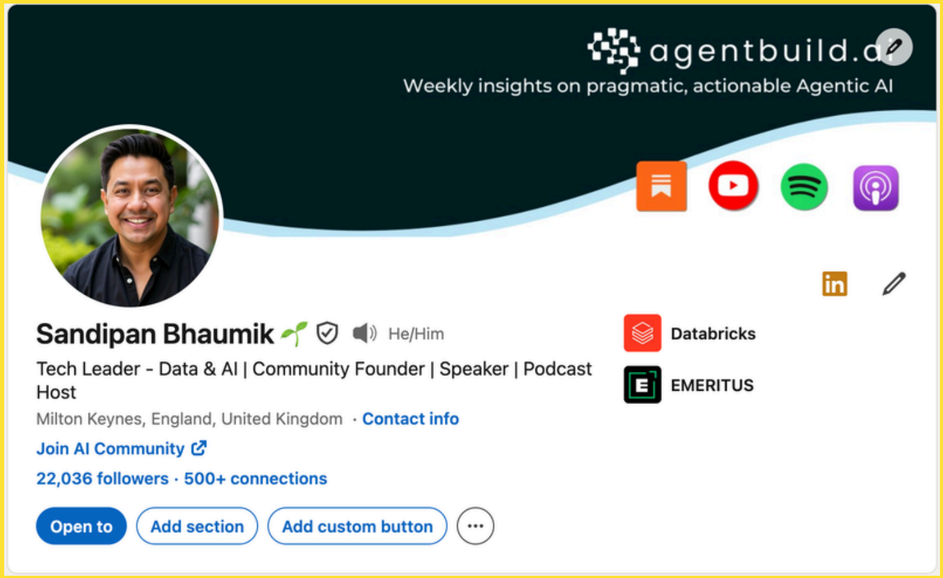
PRODUCTION


ORCHESTRATOR
Workflow Engine (DAG)
State Store (v0, v1, v2...)
Observability (Tracing)
1
state v0
Agent A
return
state v1
2
Agent B
return
state v2
2
Agent C
return
state v3
3
Agent D

PRODUCTION

DATABRICKS





SCAN QR

Agent Chaos is INEVITABLE

Choreography is a CHOICE

Build SYSTEMS, not demos